

Верификация Категорий 1 и 2 — Game Development DSS

Сводное исследование по чек-листу CATEGORY_VERIFICATION_PLAN.md

ВЕРИФИКАЦИЯ КАТЕГОРИЙ 1 И 2 — СВОДНОЕ ИССЛЕДОВАНИЕ

Проект: локальная DSS для проектирования игр (C:/Users/user/Desktop/game-inspect)

Дата: 2026-09-11 (сессия 3)

Спецификация-источник истины: F:\Downloads\важно.txt

Чек-лист параметров: research/workqueue/CATEGORY_VERIFICATION_PLAN.md

Мандат: для каждого параметра — источники, ключевые выводы, расчёты, примеры из игр; сверка с пунктами плана на правдивость и достоверность.

Как читать этот документ. Раздел 0 фиксирует механическую проверку

фундамента данных (по циклу HANDOFF §B.5) — без неё любой анализ ниже

недостоверен. Далее по одному разделу на параметр: Категория 1 (1.1–1.7,

технические параметры) и Категория 2 (2.1–2.10, параметры проектирования с

собственными расчётами). Каждый раздел заканчивается сверкой с планом.

Все числа — из production-БД, перепроверены запросом; самоотчёты не принимались.

0. ВЕРИФИКАЦИЯ ФУНДАМЕНТА (§B.5 HANDOFF)

Выполнен типовой цикл оркестратора перед исследованием:

Шаг	Проверка	Ожидалось	Факт	Итог
0	Бэкап БД	gamedev-dss.bak-*.db	gamedev-dss.bak-20260911-104519.db (5,7 МБ)	□
1	evidence_claims published	2539	2539 (всего 2562)	□
2	evidence_sources published	935	936 (всего 936; +1 — реестровый источник PassMark, регистрируется фиксом железа, см. §5)	□
3	graph_checks mandatory-циклы	0	0 (issues: 1×info version_unknown)	□
4	Тесты бэкенда	86	91 passed (86 + 5 новых, см. §5)	□
5	Аудит покрытия	—	перегенерирован; все декларации закрыты (§5)	□

Критический нюанс подтверждён: published-счётчики совпали с §B.5 (2539/935), поэтому перезагрузка БД не требовалась. **Если бы число published claims было близко к 438 — это означало бы возврат дефекта «невидимые доказательства», и первым шагом была бы перезагрузка по §B.5, а не анализ.** Анализ ниже выполнен на полной базе, а не на ~1 % данных.

Полная база (прямой запрос):

evidence_claims 2562 (published 2539, draft 23) evidence_sources 936 (published 936)
 conflicts 458 dependency_edges 929 (mandatory 164) game_cases 155 case_evidence 375
 methods 124 game_functions 40 engines 7 engine_tools 70 technology_nodes 221
 method_engine_links 473 work_packages 1794 hardware_cpu 51 hardware_gpu 79
 claims no basis: documented 1580 · case_evidence 244 · unknown 262 · derived 208 ·

expert_estimate 164 · measured 104

Ключевой маппинг «параметр → entity БД» (проверен):

Параметр плана	entity в БД	codes	published claims
1.1 Игровые функции	game_function	40	267
1.2 Методы	method	124	1177
1.3 Движки	engine	7	50
1.4 Инструменты движков	engine_tool	70	559
1.5 Конфликты	conflicts	458 строк, 7 типов	—
1.6 Связки метод-инструмент	method_engine_links	473	—
1.7 Technology nodes	technology_node	221	99
2.1 Стадии и бюджет	stage_budget	8	56
2.2 Целевые платформы	target_platform	4	24
2.3 Масштаб сцены	load_profile + функции стриминга	5 + 2	24 + ...
2.4 Сетевой режим	network_mode	6	33
2.5 Целевые показатели	target_metric	15	56
2.6 Риски	risk_factor	10	44
2.8 Профиль нагрузки	load_profile	5	24
2.9 Оборудование	hardware_cpu / hardware_gpu	51 / 79	51 / 79
2.10 Итоговый план	work_packages	1794	—

Расхождение с §B.2 HANDOFF (несущественное). В HANDOFF распределение по entity указано как method 1179 / engine_tool 580; фактический запрос к БД даёт 1177 / 559 (published-only). Разница — 2 и 21 запись (в HANDOFF часть черновиков учтена вместе с published). Все прочие счётчики совпали.

КАТЕГОРИЯ 1 — ТЕХНИЧЕСКИЕ ПАРАМЕТРЫ

Для каждого параметра: 2–3 глубоких исследования-анализа с конкретными источниками + выводы, подкреплённые примерами из разных игр (из game_cases/case_evidence, не выдуманы).

1.1 Игровые функции (game_function, 40 шт., 267 claims)

Покрытие (аудит): 40/40 функций имеют ≥ 3 claims, ≥ 2 источника с локатором, ≥ 2 игровых примера — 100 % fully-ok. Объявленных пробелов нет.

Ключевые выводы по репрезентативным функциям:

Функция	claims	Ключевой вывод (с источником)	Примеры игр
open_world_streaming	7	World Partition — автоматическое управление данными и дистанцией (Epic, SRC-FUNC-004); работает вместе с OFFPA, HLOD и Data Layers; Instanced Foliage — своя сетка 256 м (SRC-FUNC-012)	Unreal Engine City Sample (direct), Horizon Zero Dawn, No Man's Sky
advanced_npc_ai	7	F.E.A.R.: FSM из трёх состояний + GOAP; world state — массив 4-байтовых значений, что ограничивает число фактов (SRC-FUNC-052)	F.E.A.R. (direct), Left 4 Dead (AI Director)

Функция	claims	Ключевой вывод (с источником)	Примеры игр
ray_traced_effects	6	Общий лучевой конвейер у Path Tracer и real-time RT (Epic, SRC-FUNC-010); главный драйвер стоимости — расходимость лучей (NVIDIA GTC 2019, Battlefield V, SRC-FUNC-079)	Alan Wake 2, Battlefield V
crowd_simulation	7	ECS-подход: данные в кэш-дружественном виде, системы обрабатывают пачками	Assassin's Creed Unity (direct), Left 4 Dead
split_screen_rendering	11	Каждый выюпорт — отдельный полный проход сцены; нагрузка растёт линейно по выюпортам	It Takes Two (direct), Split Fiction
multiplayer_netcode	7	Дельта-сжатие состояния, приоритезация релевантности, lag compensation	Counter-Strike 2, VALORANT, Overwatch (direct x3)
audio_system	6	Окклюзия/пропагация — доминирующий драйвер стоимости аудио-бюджета	Hunt: Showdown, Returnal (direct x2)
procedural_terrain	6	Чанковая генерация; GPU-размещение растительности	No Man's Sky, Supreme Commander 2

Полный список 40 функций с покрытием приведён в приложении (см.

research/_verify_cache/digest.txt, раздел 1.1). Все функции имеют ≥ 2 примеров из разных игр.

Сверка с планом 1.1: ВЫПОЛНЕНО. Каждая функция подтверждена claims с локатором и примерами из разных игр. Пометка плана «—» (пример не указан) для части функций снята: примеры есть в case_evidence (например, crowd_simulation → Assassin's Creed Unity, audio_system → Hunt: Showdown / Returnal).

1.2 Методы (method, 124 шт., 1177 claims)

Покрытие (аудит): 100 % имеют ≥ 3 claims и ≥ 2 источника с локатором; 98,4 % имеют ≥ 2 игровых примера. Единственный метод без игрового примера — temporal_radiance_cache (объявленный пробел adoption, честно помечен adoption_evidence_gap); neural_texture_compression объявил no_shipped_title.

Классификация примеров. role: direct = shipped-тайтл на том же движке, что и метод; cross_engine = пример с другого движка (доказывает возможность, HE adoption). Методы с прямыми примерами (пример): tickrate_budgeting (3 прямых), ecs_data_oriented_crowd (2), audio_occlusion_propagation (2), npc_perception_budget (2), agent_update_budget (2), crowd_instancing_impostors (2), ability_visual_effect_budget (2), world_partition_streaming (1), virtual_geometry_clusters (1), flow_field_pathing (1) и др.

Глубокие исследования по ключевым методам:

Метод	claims	Источники (SRC-*)	Ключевой вывод	Примеры игр
world_partition_streaming	10	SRC-WRS-001	Epic-пример: Cell Size 256 м, Loading Range 768 м; важность ячейки взвешивается углом обзора, а не только дистанцией	City Sample (direct), Fortnite, UE5 Open World
motion_matching	14	SRC-CHC-010/011	Канонический алгоритм (Clavet, GDC 2016): поиск по базе записанных поз; трек-прогноз должен учитывать коллизии; Epic предупреждает не переусложнять запрос	For Honor (direct), Alan Wake 2
meshlet_pipeline_adoption	12	—	Mesh shaders + culling meshlets; Nanite даёт автоматический LOD почти на всей геометрии, но не на foliage	Alan Wake 2, Black Myth: Wukong, Hellblade 2
virtual_geometry_clusters	11	—	Нанит-кластеры; hard_conflict с gpu_instancing_vegetation: Nanite «не очень хорош с агрегатами» (трава, листья, волосы)	Lumen in the Land of Nanite (direct), City Sample
client_prediction_reconciliation	10	—	Предсказание + кольцевой буфер перемещений + сглаживание остаточной ошибки	Overwatch, VALORANT, Source-титутлы
directstorage_io	12	—	SSD-ориентированный конвейер ввода-вывода; платформенный API, а не инструмент движка	Ratchet & Clank: Rift Apart, Forspoken
tickrate_budgeting	11	SRC-DER-033/034	Бюджет тика = период/игры-на-ядро × (1-overhead); Riot: 7,8125 мс / 3 = 2,34 мс	VALORANT (direct ×2), Counter-Strike 2 (direct)
hierarchical_lod	13	—	Слияние дальних объектов; класс-сплит merged-mesh + HLOD	Horizon Zero Dawn, No Man's Sky, UE5 HLOD workflow
gpu_instancing_vegetation	11	—	GPU-риггинг растительности (AW2: ~300 000 костей)	Alan Wake 2, Horizon Zero Dawn, AC Unity
temporal_upscaling	7	—	Реконструкция 4K из меньшего внутреннего разрешения (TSR/FSR/DLSS)	Hellblade 2, Lords of the Fallen, Black Myth: Wukong, Horizon Zero Dawn
ml_frame_generation	7	—	Генерация кадров добавляет задержку; risk-связь с client_prediction_reconciliation	Alan Wake 2, Hellblade 2
screen_space_gi	7	—	Экранный GI — дешёвая альтернатива аппаратному RT-GI	Gears 5 / Hivebusters, Metro Exodus
hardware_raytraced_gi	9	—	Аппаратное RT-GI дороже screen-space; связь risk с static_shadow_caching	Alan Wake 2 (direct), Cyberpunk 2077, Metro Exodus

Сверка с планом 1.2: ВЫПОЛНЕНО. Для каждого метода есть claims с источником и ≥2 примерами (кроме 1 объявленного пробела). Пометка плана о методах nanite_geometry/lumen_gi уточнена: это коды engine_tool (ue_nanite, ue_lumen), а не методы; соответствующие методы — virtual_geometry_clusters (Nanite) и hardware_raytraced_gi/screen_space_gi

(Lumen-класс GI). Это исправление маппинга, а не пробел данных.

1.3 Движки (engine, 7 шт., 50 claims)

Покрытие: 7/7 — 100 % fully-ok (≥3 claims, ≥2 источника, ≥2 примера).

Движок	claims	Ключевые факты (SRC)	Игровые примеры
unreal	7	C++ (SRC-ENG-052); роялти 5 % свыше \$1M; UE5 — апрель 2022; Windows/Linux/macOS; Fortnite: ~100 игроков, ~50 000 реплицируемых акторов (SRC-ENG-017)	Fortnite, проект на Nanite+Lumen (SRC-ENG-067)
unity	7	Runtime на C++, API на C# (SRC-ENG-049); Personal free при выручке <\$200K; 25+ платформ; Unity 6 (6000.6) (SRC-ENG-027); V Rising на DOTS+HDRP — 5M+ копий, 60 игроков, 1 600 ECS-систем (SRC-ENG-045)	V Rising, DOTS-проект (1 ч → 100 мс, SRC-ENG-043)
godot	7	GDScript/C#/C++/GDExtension; MIT (SRC-ENG-055); Godot 4.7.2 + LTS 3.6.x (SRC-ENG-053); консоли только через сторонних издателей	Brotato, Cassette Beasts
cryengine	8	C++/Lua/C# (SRC-ENG-059); 2002 → 5.7.1 LTS; Windows/Linux/консоли; Far Cry (2004), Crysis (2007)	Crysis, Kingdom Come (модиф. CryEngine)
source	6	C++ (SRC-ENG-060); Source SDK; 2004 → Source 2 (2015); Source 2: Dota 2, HL:Alyx, CS2, Deadlock (SRC-ENG-061)	Half-Life 2, Counter-Strike 2
heroengine	8	C++/C#/HSL (SRC-ENG-063); HeroCloud \$99/год + 30 % revenue share; 2.074; Windows/macOS; MMO-платформа	ММО на HeroEngine (5+ лет разработки)
custom	7	In-house на ECS-ядре; Bevy — ECS-first, Godot на C++ (SRC-ENG-054/055); RAGE — проприетарный in-house Rockstar (SRC-ENG-065); tool-level документации нет → unknown (SRC-ENG-056)	GTA V, Red Dead Redemption (RAGE)

Сверка с планом 1.3: ВЫПОЛНЕНО. Пометка плана «Bevy — reference» подтверждена:

Bevy фигурирует как reference-реализация (не shipped-игра), честно помечено.

1.4 Инструменты движков (engine_tool, 70 шт., 559 claims)

Покрытие (аудит): 70/70 — 100 % (≥3 claims, ≥2 источника, ≥2 примера

включая cross-engine). Строгий показатель: entities_with_direct_proof = 52

из 70 (прямой пример = shipped-тайтл на том же движке); 18 — только перекрёстные;

21 объявил пробел adoption (adoption_evidence_gap); 0 молчаливых дыр

(entities_unproven_and_undeclared = 0).

Префиксы: ue_* (Unreal, 27 инструментов), u_* (Unity, 15), g_* (Godot,

13), s_* (Source, 5), ce_* (CryEngine, 4), h_* (HeroEngine, 4), c_*

(CryEngine/reference-реализации, 7 — HE игры).

Примеры глубокого разбора:

Инструмент	claims	Ключевой вывод	Пример
ue_nanite	9	Виртуализированная геометрия: автоматический LOD без pop-in	Hellblade 2, Black Myth: Wukong
ue_lumen	9	Динамическое GI + отражения в реальном времени	Alan Wake 2
ue_world_partition	9	Ячейки + OFPA + Data Layers + HLOD	City Sample, Fortnite
ue_replication_graph	10	Граф репликации с приоритетом/дормантностью/релевантностью	Fortnite (100 игроков, 50k акторов)
g_gi	10	Godot: встроенное GI-решение	Godot-based тайтлы (engine capability)
c_ecs	8	Reference: bevy_ecs; EnTT используется в Minecraft (Mojang) и ArcGIS (SRC-GCS-062)	Minecraft (EnTT)
c_profiler	8	Tracy — ns-разрешение, remote telemetry; VProf у Valve (SRC-GCS-048/058)	Reference, не игра
c_manual	9	The Forge намеренно не даёт физику/сеть/звук — всё вручную (SRC-GCS-061)	Reference, не игра

Сверка с планом 1.4: ВЫПОЛНЕНО. Пометка «52/70 с прямым примером; 18 только перекрёстных; 21 пробел» подтверждена точно. с_* честно отделены как reference-реализации (не игры).

1.5 Конфликты зависимостей (conflicts, 458 строк, 7 типов)

Распределение по типам (прямой запрос):

Тип связи	Строк	Смысл
complement	183	Взаимодополнение (взаимное)
dependency	80	Обязательное предусловие
risk	75	Совместимо, но есть риск качества/производительности
overlap	67	Пересекающаяся область (часто симметрично)
alternative	41	Взаимозаменяемые
unknown	8	Отношение не классифицировано
hard_conflict	4	Жёсткая несовместимость (взаимоисключение)

Все 4 hard_conflict (разобраны выборочно, severity 3):

А	В	Документированная причина
motion_matching	animation_lod_budget	Root motion обязателен для UE motion matching, но блокирует параллельное обновление анимации → теряется масштабирование главного потока
portal_scene_capture_budget	splitscreen_render_budget	Оба умножают число проходов сцены; split screen + порталы — худший случай бюджета кадра
baked_occlusion_culling	world_partition_streaming	Предвычисленная видимость «плохо работает со стримингом уровней: все данные лежат в persistent-уровне»
virtual_geometry_clusters	gpu_instancing_vegetation	Nanite «не очень хорош с агрегатами» (трава/листья/волосы) → foliage нужен отдельный instancing-путь

Примеры risk (severity 2-3): deterministic_lockstep ↔

multithreaded_physics_jobs (детерминизм требует фиксированного порядка);
 ml_frame_generation ↔ client_prediction_reconciliation (генерация кадров
 добавляет задержку и артефакты пересборки); runtime_fracture_budget ↔
 baked_occlusion_culling (изменение геометрии обесценивает предвычисленную
 видимость).

Примеры dependency: animation_compression → motion_matching;
 build_size_startup_budgets → directstorage_io;
 client_prediction_reconciliation → tickrate_budgeting.

Сверка с планом 1.5: ВЫПОЛНЕНО. План предлагал разобрать 5–10 конфликтов —
 разобраны все 4 hard_conflict + 3 risk + 3 dependency с источниками и
 примерами. Все 7 типов присутствуют и спец-проверены в спец-compliance-matrix.md.
 Замечание: 30 строк conflicts без source_url (это выведенные
 dependency-связи «А требует В» из каталога методов; см. §5).

1.6 Связки «метод-инструмент» (method_engine_links, 473)

Состояние: 473 связки; 362 имеют source_url, 111 — без URL
 (реализуются «своими средствами»). Примеры текстов заметок без URL: «Подсистему
 стриминга необходимо реализовать самостоятельно», «Реализуется полностью
 самостоятельно», «Реализуется как проход собственного графа рендера».
 Ключевой вывод. Связка «метод → инструмент» подтверждает, что метод
 реализуется заявленным инструментом, только при наличии source_url; 111 связок
 сознательно описывают отсутствие инструмента (собственная реализация).
 Сверка с планом 1.6: ВЫПОЛНЕНО (исправлено). Изначально evidence_basis у
 всех 473 связок был unknown, а объявление «своя реализация» жило только в
 свободном тексте note — аудит фиксировал undeclared_missing_url = 111.
 Дефект был вызван порядком заполнения: проход нормализации выполнялся до
 создания самих инструментов и связок. Исправлено функцией
 declare_evidence_gaps() (см. §5): инструменты собственной реализации помечены
 is_user_defined, 111 связок получили evidence_status='user_defined' и
 локатор. Итог аудита: pct_ok = 100 %, undeclared_missing_url = 0.

1.7 Technology nodes (technology_node, 221, 99 claims)

Состав: method 124 · tool 70 · plugin 8 · engine 7 · lib 6 · api 4 · sdk 2.
 Покрытие (аудит): pct_fully_ok = 97,3 %; из 20 автономных узлов (не
 унаследованных из каталога) — 80 % fully-ok; 4 объявили отсутствие shipped-доказательства,
 4 «документированы, но не доказаны» — честно.
 Примеры автономных узлов:

Узел	Тип	Документация	Пример использования
api:directx12	api	DX12, SM6 (SRC-DER-007/008)	Windows-тайтлы
api:vulkan	api	C99-API для низкоуровневой графики/вычислений (SRC-DER-021)	Linux/Deck

Узел	Тип	Документация	Пример использования
sdk:directstorage	sdk	Microsoft DirectStorage	Forspoken, Ratchet & Clank
sdk:steamworks	sdk	Steamworks SDK (SRC-DER-024/025)	P2P-релеи Valve
plugin:ue.world_partition	plugin	World Partition / OFPA	City Sample
plugin:ue.replication_graph	plugin	Replication Graph	Fortnite
plugin:unity.entities	plugin	Unity Entities (DOTS)	V Rising
lib:meshoptimizer	lib	Оптимизация меша (объявлен пробел shipped-proof)	—
lib:acl	lib	Animation Compression Library (пробел)	—

Сверка с планом 1.7: ВЫПОЛНЕНО. Для узлов есть документация/спецификация (api_specification 5, official_documentation 317) и примеры; 4 автономных узла честно объявляют отсутствие shipped-доказательства.

КАТЕГОРИЯ 2 — ПАРАМЕТРЫ ПРОЕКТИРОВАНИЯ

Для каждого: глубокие исследования + **собственное исследование проекта с расчётами** (пример на игре недостаточен для доказательства корректности внутри проекта). Источник расчётов — 208 derived claims с formula + input_parameters (в БД и research/packs/pack_derivations.json), пересчитанных и подтверждённых заново.

2.1 Стадии и бюджет (stage_budget, 8 стадий, 56 claims)

Ключевые выводы. 8 стадий (concept → post_release), у каждой: exit_criteria (expert_estimate) и late_introduction_multiplier (derived, SRC-DER-036). Модель опирается на Бозма (стоимость исправления дефекта растёт от фазы «требования» к «эксплуатации»): design 3–8×, code 5–20×, dev-testing 10–50×, acceptance 30–100×, operations 50–200×.

Стадия	exit_criteria (кратко)	risk_of_rework
concept	Питч с платформой, FPS/разрешением и объёмом утверждён	high
preproduction	Vertical slice на минимальной машине; риски перечислены	high
prototype	Throw-away билд доказывает рискованное допущение	high
production	Content-complete против замороженного feature-list	medium
alpha	Feature-complete; бюджет кадра в автотестах	low
beta	Content/feature-locked; сертификация	low
release	Билд подписан; store и defaults live	very low
post_release	Live-ops SLO; save-совместимость	very low

Собственное исследование (расчёт):

$\text{late_introduction_multiplier_x} = \text{relative_cost_to_fix}(\text{phase_found}) / \text{relative_cost_to_fix}(\text{requirements})$

(вход: фаза обнаружения дефекта). Проверка: множитель монотонно растёт по стадиям, что согласуется с quality-гейтами alpha/beta (баг, найденный в beta,

стоит 30–100× дефекта фазы требований).

Примеры игр. Anthem — попытка re-scope после релиза (risk scope_creep, SRC-DER-032); Godot 3→4 миграции — риск late_technology_adoption (SRC-DER-011).

Сверка с планом 2.1: ВЫПОЛНЕНО. Бюджет стадии и множитель позднего внедрения присутствуют; сверено с work_packages (1794, см. 2.10).

2.2 Целевые платформы (target_platform, 4, 24 claims)

Ключевые выводы. pc_windows — основной количественный таргет: DX12-класс GPU у 91,45 % Steam-систем; Windows 11 70,97 % + Win10 22,90 % ≈ 94 % (SRC-DER-001/002). pc_linux — Vulkan + Steam Deck (4–15 Вт APU, Proton, SRC-DER-021/022). **console_reference и mobile_reference явно OUT OF SCOPE количественной PC-модели** (SRC-DER-006/014): мобильные бюджеты Epic (≤ 700 draw calls, $\leq 500k$ треугольников) даны как ориентир, но не применяются к PC. Собственное исследование. Проверено, что оценка **не применяет PC-числа к консоли/мобайлу**: консоль/мобайл помечены out_of_scope и не участвуют в required_cpu_index/required_gpu_index PC-модели. Это соответствует spec («non-PC цели не попадают в количественную PC-модель»).

Примеры игр. Godot 4 миграция шейдеров (SRC-DER-011, Linux/Deck);

Steam Deck: 30 fps default при 800p (SRC-DER-023).

Сверка с планом 2.2: ВЫПОЛНЕНО.

2.3 Масштаб сцены (load_profile + open_world_streaming/large_scale_terrain)

Исследование. Поточковая генерация (City Sample, No Man's Sky, Horizon Zero Dawn): размер мира задаёт стриминг, резидентную память и объём контента.

Собственное исследование (важнейший инвариант, mutation-проверен).

Доказано, что масштаб сцены меняет нагрузку/стриминг, но НЕ стоимость кадра (тест test_scene_scale_changes_load_not_frame_cost). Живой прогон на production-БД (функции open_world_streaming + crowd_simulation, 1080p/high/60 fps):

Показатель	small	very_large	Δ
RAM, ГБ	11,5	15,2	↑
VRAM, ГБ	6,4	9,4	↑
draw calls	23 567	41 825	↑
класс накопителя	sata_ssd	nvme	↑
required_cpu_index	0,284	0,284	=
required_gpu_index	0,213	0,213	=

Вывод. Размер мира = стриминг/резидентность/объём контента; работу на кадр задаёт активная сцена (объекты/NPC). Ранее единый множитель content молча удорожал кадр от площади мира — исправлено и закреплено тестом.

Примеры игр. No Man's Sky, Horizon Zero Dawn, City Sample, Minecraft.

Сверка с планом 2.3: ВЫПОЛНЕНО (инвариант подтверждён живьём; мутация world = content поняет тест).

2.4 Сетевой режим (network_mode, 6, 33 claims)

Исследование. Netcode: Valorant (128 Гц), CS2 (64 тика, sub-tick), L4D (30 тиков), Source (66,67 Гц / 15 мс). dedicated_server: тик 30–128 Гц, $\text{players_per_core} = (\text{games_on_host}/\text{cores}) \times \text{players_per_game}$ (SRC-DER-033).

Собственное исследование (расчёты, пересчитаны):

Расчёт	Формула	Значение (вход)	Источник
Сетевой бюджет	$\text{clients} \times \text{updates/s} \times (\text{payload} + \text{ipv4/udp header})$	26 640 Б/с $\approx$ 213 kbps (9 клиентов, 20 апд/с, 120 Б + 28 Б)	SRC-DER-004 / RFC 9000
Период тика	$1000 / \text{tick_hz}$	7,8125 мс (128 Гц)	SRC-DER-033 / Riot
Бюджет сервера на игру	$(1000/\text{tick}) / \text{games_per_core} \times (1 - \text{overhead})$	2,34 мс (7,8125 / 3 $\times$ 0,9)	SRC-DER-033
P2P-связи	$n \times (n-1) / 2$	квадратичный рост	SRC-DER-026
Задержка коррекции	$2 \times \text{one_way_latency}$	200 мс при 100 мс в одну сторону	SRC-DER-004

Примеры игр. VALORANT (128 Гц), Counter-Strike 2 (64 tick + sub-tick), Left 4 Dead 2 (30 tick), Quake (уход от P2P-lockstep).

Сверка с планом 2.4: ВЫПОЛНЕНО. Сетевой бюджет и период тика пересчитаны под целевой tickrate.

2.5 Целевые показатели качества/производительности (target_metric, 15, 56 claims)

Собственное исследование (расчёты, перепроверены):

Расчёт	Формула	Значение	Источник
Бюджет кадра (60 fps)	$1000 / \text{target_fps}$	16,6667 мс	SRC-DER-035
Бюджет кадра (30 / 120 / 144)	$1000 / \text{fps}$	33,33 / 8,33 / 6,94 мс	SRC-DER-035/017/034
Pixel ratio 4K vs 1080p	$(3840 \times 2160) / (1920 \times 1080)$	4,0	SRC-DER-001
Закон Амдала ($s=0,40$, $N=8$)	$(Ts+Tp)/(Ts+Tp/N)$	2,1053	SRC-DER-037 (Amdahl)
1% low (ранг)	$\text{ceil}(0,99 \times N)$	ранг 9 900 при $N=10\,000$	SRC-DER-019/020
1% low (FPS)	$1000 / p99_frame_time_ms$	обратная величина p99	SRC-DER-020
VRAM-пол по разрешению	$w \times h \times \text{bpp} \times \text{targets} / 1048576$	—	SRC-DER-009
Save-столл	$(\text{save_bytes}/1e6)/\text{write_MBps} \times 1000$	столл $\geq$ 6 кадров при 100 мс	SRC-DER-018/019

Ключевые выводы. Бюджет кадра — потолок для CPU и GPU одновременно, а не цель по среднему. 1% low — перцентиль времени кадра, а не FPS. Целевой набор PC: 30/60/120 fps; Steam Deck: 30 fps при 800p.

Примеры игр. VALORANT (144 fps — конфигурация для моделирования peeker's advantage), CS:GO (64 vs 128 tick).

Сверка с планом 2.5: ВЫПОЛНЕНО.

2.6 Риски проекта (risk_factor, 10, 44 claims)

Исследование. Postmortem-ы (source_type=postmortem), IGDA DSS 2023 (SRC-DER-031): 28 % респондентов сообщили о crunch + 25 % — о «crunch-подобных» периодах; среди испытывших crunch 63–75 % — повторно.

Собственное исследование (расчёты):

Риск	Расчёт	Значение
Оценка длительности (PERT)	$(O + 4M + P) / 6$	4,6667 чел-дней (2, 4, 10) — SRC-DER-035/PMI
Координационные издержки (scope_creep)	$n \times (n-1) / 2$	квадратично (Брукс, 50 чел → 1 225 каналов)
Impact-полоса	$relative_cost_to_fix(phase)/relative_cost_to_fix(req)$	Боэм-множители
Productivity spread (crunch)	best / median	5-10× (SRC-DER-026)

Все 10 рисков: scope_creep, late_technology_adoption, content_scale_underestimate, performance_regression, platform_api_breakage, dependency_abandonment, multiplayer_scope, tooling_gap, save_compatibility, crunch_burnout — у каждого trigger / detection_signal / mitigation / impact_band; probability_band честно помечен unknown, где базовой ставки нет в публичных данных.

Примеры игр. Anthem (scope_creep, SRC-DER-032); Godot 3→4 (late_technology_adoption, dependency_abandonment — Bullet→GodotPhysics); crunch/burnout — доказательство опросное (IGDA DSS 2023), игрового примера намеренно нет.

Сверка с планом 2.6: ВЫПОЛНЕНО. PERT = 4,6667 чел-дней подтверждён.

2.7 Корзины (baskets) — связь с method_engine_links + recommend API

Исследование. Методы группируются в рабочие корзины; инвариант: ****hard conflict не попадает в корзину****. Проверено через recommender + method_engine_links.

Собственное исследование. Проверено, что recommend не предлагает несовместимое: при наличии hard_conflict между двумя методами второй не включается в рекомендованную корзину; исключённые/вне-кадра эффекты ****названы в contributions.exclusions****, а не пропущены молча (тест test_out_of_frame_effect_is_named_not_silent).

Примеры игр. Корзина «рендеринг открытого мира»: City Sample, Fortnite.

Сверка с планом 2.7: ВЫПОЛНЕНО (спес-инвариант подтверждён тестом).

2.8 Профиль нагрузки (load_profile, 5, 24 claims)

Ключевые выводы. 5 профилей: client_runtime, server_runtime, editor_workflow, build_cook, development_workstation. Клиент — много-поточный, но не «embarrassingly parallel» (6 потоков UE, SRC-DER-015).

Сервер — узкое место: покадровое время одного потока, затем конкуренция за кэш (Riot: 1,5 мс/кадр при 1 инстансе, SRC-DER-033).

Собственное исследование (важный инвариант, mutation-проверен).

Область эффекта (effect_scope): серверная экономия не удешевляет клиент.

Живой прогон (функция multiplayer_netcode, scale large):

Конфигурация	`required_cpu_index`	`required_gpu_index`
Базовая (только клиент)	0,284	0,151

Конфигурация	`required_cpu_index`	`required_gpu_index`
+ корзина headless_dedicated_server	0,284	0,151

Вывод. Headless-сервер не улучшает графику игрока: серверный эффект не применяется к клиентскому PC (тест test_server_effect_does_not_discount_player_pc).

Примеры игр. VALORANT (клиент 128 Гц; сервер: анимация –75 % каждым 4-м кадром, ещё –33 % в buy-фазе), L4D (30 tick).

Сверка с планом 2.8: ВЫПОЛНЕНО.

2.9 Оборудование (hardware_cpu 51 / hardware_gpu 79)

Ключевые выводы. Нормализованные индексы 0..1 из benchmark (PassMark, source_type=hardware_benchmark 129). Диапазоны CPU: Ryzen 3 1200 (0,41) → i9-14900K (1,00); GPU: GTX 1050 Ti (0,13) → RTX 40-серия.

Собственное исследование (расчёты):

Расчёт	Формула	Значение
Плановый headroom памяти	$(3,5 + 1,4 + 0,2) \times 1,20$	6,12 GiB
required_gpu_index	сумма подсистем, нормализация	из активной цены
required_cpu_index	сумма подсистем, нормализация	из активной цены

Аудит: 0 записей без benchmark_context, evidence_basis, normalization_note, source_url. Замечание: поле benchmark_raw_value пусто у всех (51/79) — сырое значение хранится в других полях/ноте нормализации; формального нарушения нет, но рекомендуется заполнить для прослеживаемости. Примеры. Steam-модаль: 16 ГБ RAM (41,2 %), 8 ГБ VRAM (25,7 %); RTX 4060 — 8 ГБ GDDR6/128 бит (SRC-DER-028).

Сверка с планом 2.9: ВЫПОЛНЕНО (память 6,12 GiB подтверждена).

2.10 Итоговый план (work_packages, 1794)

Ключевые выводы. 1794 пакета работ, все с basis=expert_estimate; сумма P50 = 3511,8 чел-дней; 1422 параллелизуемых. Стадии: prototype 741, production 656, preproduction 367, concept 22, alpha 8.

Собственное исследование (живой прогон планировщика на production-БД).

Корзина (10 валидных кодов методов): world_partition_streaming, virtual_geometry_clusters, hardware_raytraced_gi, temporal_upscaling, virtual_shadow_maps, motion_matching, client_prediction_reconciliation, tickrate_budgeting, directstorage_io, gpu_compute_culling. Профиль: multiplayer, open_world, large. Планировщик раскрыл корзину с обязательными условиями до 21 метода (139 задач, 0 неразрешённых зависимостей).

Команда	`effort.p50` (чел-дни)	`calendar.p50` (дни)	задач	методов	критических
solo	208,99	190,12	139	21	7
small_2_5	208,99	116,58	139	21	7
mid_6_15	208,99	52,59	139	21	7
large_16_plus	208,99	25,12	139	21	7

Инвариант подтверждён живьём: effort (человеко-дни) не меняется от размера команды (208,99 при любом размере); календарь падает 190,12 → 25,12 дней (solo → large, сжатие ×7,6). Критический путь — 7 задач world_partition_streaming (design→feasibility→integration→content→optimization→qa→release), каждая помечена critical, что совпадает с множеством critical_path.

Примечание о кодах. Исходный список плана содержал nanite_geometry,
lumen_gi, gpu_driven_rendering — это коды engine_tool, а не method
(проверено запросом: entity='method' их не содержит). В прогоне использованы
реальные методы virtual_geometry_clusters (Nanite),
hardware_raytraced_gi (Lumen-класс GI), gpu_compute_culling. Это уточнение
маппинга, а не изменение результата.

Иллюстративный расчёт длины пути: $\max(3+5+6+4, 3+8+6+4) = 21$ день (SRC-DER-035, NASA schedule risk).

Сверка с планом 2.10: ВЫПОЛНЕНО. Инвариант «team size ≠ person-days» подтверждён (мои живые числа: 208,99 чел-дней при любом размере команды; календарь 190,12 → 25,12).

СВОДКА СВЕРКИ С ПЛАНОМ

№	Параметр	Источники	Выводы	Расчёт	Примеры игр	Сверка
1.1	Игровые функции (40)	□	□	—	□	ВЫПОЛНЕНО
1.2	Методы (124)	□	□	—	□	ВЫПОЛНЕНО
1.3	Движки (7)	□	□	—	□	ВЫПОЛНЕНО
1.4	Инструменты (70)	□	□	—	□	ВЫПОЛНЕНО
1.5	Конфликты (458, 7 типов)	□	□	—	□	ВЫПОЛНЕНО
1.6	Связки метод-инструмент (473)	□	□	—	—	ВЫПОЛНЕНО
1.7	Technology nodes (221)	□	□	—	□	ВЫПОЛНЕНО
2.1	Стадии и бюджет	□	□	□	□	ВЫПОЛНЕНО
2.2	Целевые платформы	□	□	□	□	ВЫПОЛНЕНО
2.3	Масштаб сцены	□	□	□ (live)	□	ВЫПОЛНЕНО
2.4	Сетевой режим	□	□	□	□	ВЫПОЛНЕНО
2.5	Целевые показатели	□	□	□	□	ВЫПОЛНЕНО
2.6	Риски проекта	□	□	□	□	ВЫПОЛНЕНО
2.7	Корзины	□	□	□	□	ВЫПОЛНЕНО
2.8	Профиль нагрузки	□	□	□ (live)	□	ВЫПОЛНЕНО
2.9	Оборудование	□	□	□	□	ВЫПОЛНЕНО
2.10	Итоговый план	□	□	□ (live)	□	ВЫПОЛНЕНО

Итого: 17 из 17 параметров — ВЫПОЛНЕНО. Ранее единственный пункт «ТРЕБУЕТ ДОРАБОТКИ» (1.6) закрыт исправлением деклараций (см. §5).

5. ИСПРАВЛЕННЫЕ ПРОБЕЛЫ И ЧЕСТНЫЕ ОГРАНИЧЕНИЯ

Мандат требует правдивости: пробелы сначала были перечислены, затем — устранены. Ниже — что именно было не так, что исправлено и как это закреплено, чтобы дефект не вернулся.

5.1 Что исправлено (сессия 3)

Корневая причина всех пяти пунктов одна: ****проходы нормализации деклараций выполнялись в seed_methods — до того, как создавались инструменты, связи, конфликты, рёбра графа и железо.**** Поэтому на свежей базе они не срабатывали, и декларации появлялись только при повторном заполнении. Добавлена функция `declare_evidence_gaps(db)` (`backend/app/seed/corrections.py`), вызываемая в `seed_all` последней — после построения графа.

№	Было	Исправление	Проверка (аудит)
1	method_engine_links: 111 связок без URL и без пометки	инструменты собственной реализации (custom) помечены <code>is_user_defined</code> ; 111 связок → <code>evidence_status='user_defined'</code> + локатор	<code>pct_ok</code> 100 %, <code>undeclared_missing_url</code> 0
2	conflicts: 30 строк без URL	<code>source_url='user_defined:catalog_dependency'</code> (связь выведена из каталога, не из документа)	<code>missing_url</code> 0, <code>undeclared</code> 0
3	dependency_edges: 24 ребра без декларации	префикс <code>[expert_estimate:no_external_source]</code> в описании (плановая зависимость)	<code>declared_expert_estimate</code> 154, <code>undeclared</code> 0
4	hardware_cpu/gpu: benchmark_raw_value пуст (51/79)	<code>apply_hardware_raw_values</code> вызвана после создания железа; 130 строк заполнены (<code>measured</code> или <code>derived</code> от нормализованного индекса). Побочно регистрируется реестровый источник <code>PASSMARK_2026_09</code> → <code>sources</code> 935 → 936	<code>missing_benchmark_raw_value</code> 0
5	3 «висячих» <code>derived-claim</code> (<code>research:*</code>)	аудит приведён к правилу спеки (строка 412): <code>derived c formula+input_parameters</code> самообоснован, а не «висячая ссылка»	<code>dangling claims</code> 0

5.2 Закрепление (чтобы не вернулось)

- Регрессионный тест `backend/tests/test_evidence_declarations.py` (5 тестов): связи без URL, конфликты без URL, рёбра без источника, сырые значения бенчмарков, отсутствие висячих публичных утверждений.
- Mutation-проба: при отключении `declare_evidence_gaps` 4 из 5 тестов падают (невхолостую), затем изменение откатано.
- Воспроизводимость: пересборка БД с нуля (`seed_all` на чистой БД) даёт те же декларации нативно — 111 `user_defined`, 0 конфликтов без URL, 0 рёбер без

декларации, 0 пустых сырых значений; `validation_issues` = 0.

- Тесты: 86 → 91 passed. Граф: 0 ошибок, 0 mandatory-циклов.

5.3 Оставшиеся честные ограничения (не пробелы данных)

1. `sources.missing_published_date` = 28 — у 28 источников нет даты публикации; дата не выдумывается (принцип #3). Часть — «living documentation» (декларировано отдельно). Информационная метрика, не влияющая на выводы.
2. Объявленные пробелы adoption (честно, не скрыты): метод `temporal_radiance_cache` (0 игровых примеров), `neural_texture_compression` (`no_shipped_title`), 21 инструмент движка (`adoption_evidence_gap`), 4 автономных `technology_node`. Это корректное состояние «доказательства нет», а не «дыра в данных».
3. `benchmark_raw_value` для не-якорных записей выведен из нормализованного индекса (`raw` = `normalized` × `anchor`), с явной пометкой `evidence_basis='derived'` и допуском ±5 % в `normalization_note`; для якорных записей — `measured`.

6. МЕТОДИКА И ВОСПРОИЗВОДИМОСТЬ

- Извлечение: `tools/extract_category_evidence.py` → `research/_verify_cache/category_evidence.json` (2539 published claims, 375 case_evidence, 123 метода и 40 функций с примерами).
- Дайджест: `tools/dump_digest.py` → `research/_verify_cache/digest.txt`.
- Живые инварианты (2.3, 2.8, 2.10) пересчитаны через сервисы `planning.schedule` / `hardware.estimate_hardware` на production-БД.
- Аудит покрытия: `tools/audit_evidence.py` → `research/audit_evidence.json`.
- Исправление деклараций: `backend/app/seed/corrections.py::declare_evidence_gaps` (вызывается последней в `seed_all`); регрессия — `backend/tests/test_evidence_declarations.py` (5 тестов, mutation-проверены).
- Правило: ни одно число не взято из самоотчёта — только прямой запрос к БД, прогон сервиса или тест.